

 UNIVERSIDAD DON BOSCO	UNIVERSIDAD DON BOSCO FACULTAD DE INGENIERÍA ESCUELA DE COMPUTACIÓN		CICLO 02-2026
	MATERIA	DESARROLLO DE SOFTWARE PARA MOVILES	GUIA DE LABORATORIO #11
	PRÁCTICA	CONSUMO DE API REST CON RETROFIT – PARTE II	

I. OBJETIVOS

- Conocer la librería Retrofit para enviar datos y actualizar, agregar y eliminar además de consultar información.
- Consumir un API REST usando Retrofit en una aplicación Android.

II. INTRODUCCIÓN TEORICA

En la guía anterior se abordó la forma de consumir una API usando Retrofit para consultar información usando el verbo GET.

En la presente guía se desarrollará una aplicación que hace uso de los 4 verbos para permitir hacer un CRUD. Los cuatro métodos HTTP principales que sustentan las API REST para interactuar con recursos son:

1. GET (Obtener):

Este método se utiliza para recuperar datos de un recurso específico. Comúnmente se emplea para buscar información sobre entidades existentes, como obtener una lista de usuarios o un producto específico por ID.

2. POST (Crear):

Este método se utiliza para crear un nuevo recurso. A menudo se utiliza para enviar datos al servidor, como crear una nueva cuenta de usuario o agregar un nuevo elemento a un carrito de compras. Los datos para el nuevo recurso se envían típicamente dentro del cuerpo de la solicitud.

3. PUT (Actualizar):

Este método actualiza un recurso completo. Reemplaza el recurso existente en el punto final especificado con los datos enviados en el cuerpo de la solicitud. Las solicitudes PUT generalmente se consideran idempotentes, lo que significa que llamarlas varias veces con los mismos datos producirá el mismo resultado.

4. DELETE (Eliminar):

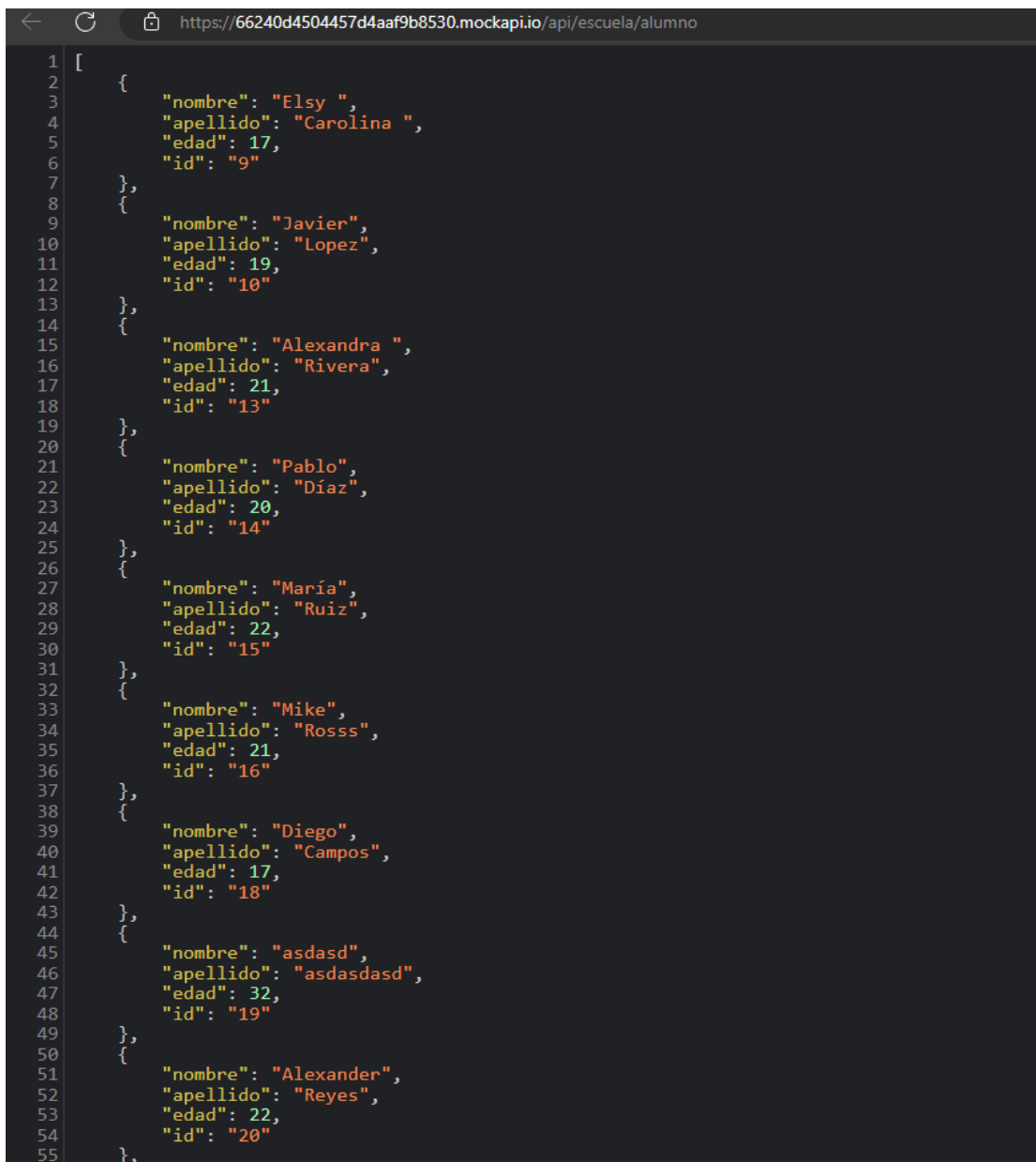
Este método elimina un recurso. Se utiliza para eliminar un recurso específico del servidor, como eliminar una cuenta de usuario o un producto. Las solicitudes DELETE no son idempotentes, ya que llamarlas varias veces resultará en múltiples eliminaciones.

En resumen, estos cuatro métodos HTTP fundamentales (GET, POST, PUT y DELETE) forman la base de las interacciones con recursos en las API REST, permitiendo a los desarrolladores crear interfaces eficientes y escalables para aplicaciones web y servicios.

Para esta guía se utilizará el recurso gratuito que está en internet: <https://mockapi.io> que permite crear un proyecto y dos recursos de manera gratuita.

MockAPI.io es una plataforma que permite crear APIs REST de forma rápida y sencilla, sin necesidad de escribir código. Es una herramienta muy útil para desarrolladores que necesitan probar sus aplicaciones o interfaces de usuario sin tener que depender de datos reales o APIs externas.

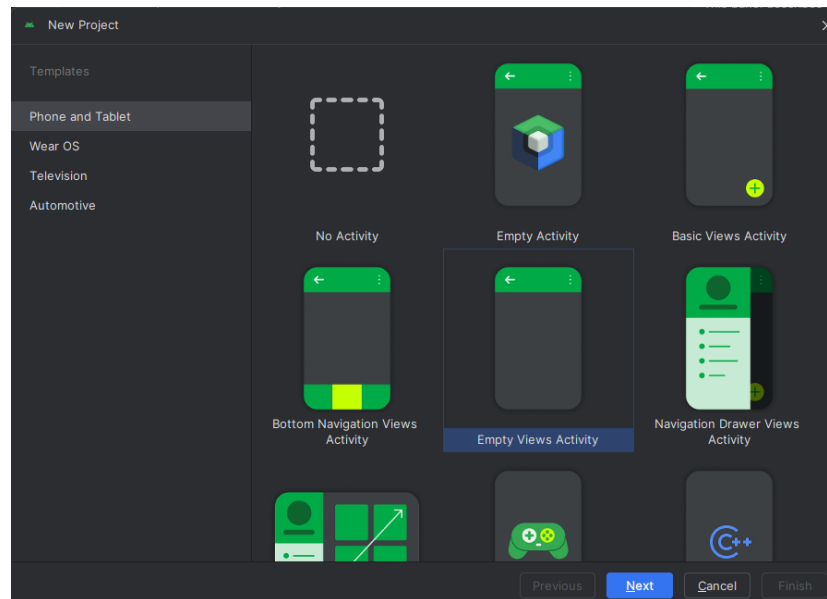
El api REST a utilizar para la práctica: <https://66240d4504457d4aaf9b8530.mockapi.io/api/escuela/alumno>



```
1 [
2   {
3     "nombre": "Elsy ",
4     "apellido": "Carolina ",
5     "edad": 17,
6     "id": "9"
7   },
8   {
9     "nombre": "Javier",
10    "apellido": "Lopez",
11    "edad": 19,
12    "id": "10"
13  },
14  {
15    "nombre": "Alexandra ",
16    "apellido": "Rivera",
17    "edad": 21,
18    "id": "13"
19  },
20  {
21    "nombre": "Pablo",
22    "apellido": "Díaz",
23    "edad": 20,
24    "id": "14"
25  },
26  {
27    "nombre": "María",
28    "apellido": "Ruiz",
29    "edad": 22,
30    "id": "15"
31  },
32  {
33    "nombre": "Mike",
34    "apellido": "Rosss",
35    "edad": 21,
36    "id": "16"
37  },
38  {
39    "nombre": "Diego",
40    "apellido": "Campos",
41    "edad": 17,
42    "id": "18"
43  },
44  {
45    "nombre": "asdasd",
46    "apellido": "asdasdasd",
47    "edad": 32,
48    "id": "19"
49  },
50  {
51    "nombre": "Alexander",
52    "apellido": "Reyes",
53    "edad": 22,
54    "id": "20"
55  },
56 ]
```

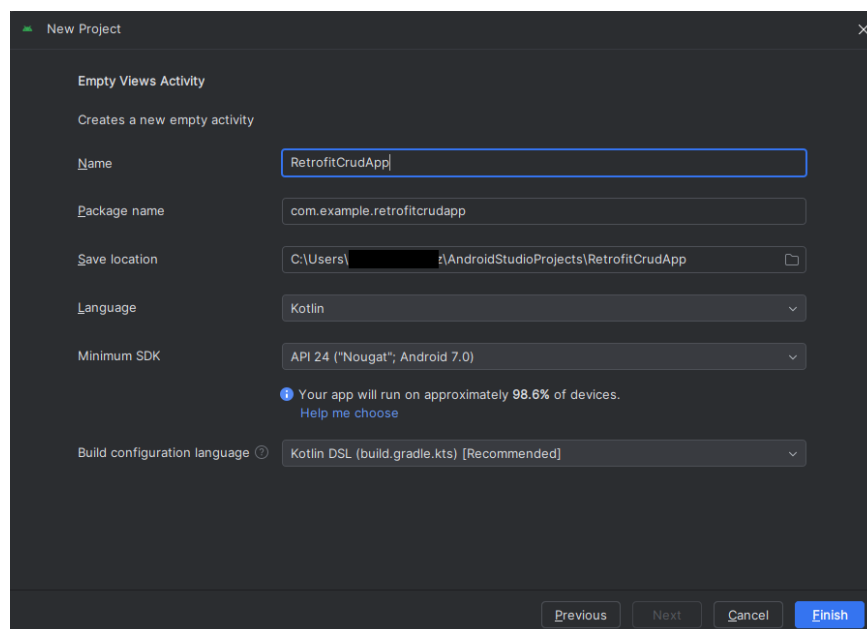
III. DESARROLLO DE LA PRÁCTICA

1. Cree un nuevo proyecto en Android Studio, asegúrese de seleccionar la plantilla **Empty Views Activity**.



2. Configure el proyecto con las siguientes características:

- **Nombre:** RetrofitCrudApp.
- **Ubicación:** por defecto (o use una ubicación de su preferencia).
- **Lenguaje:** Kotlin.
- **Minimun SDK:** API 24 Nougat.



3. Edite el archivo **AndroidManifest.xml** para agregar el permiso de conectividad a internet.

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <uses-permission android:name="android.permission.INTERNET"/>

    <application>
        .....
    </application>

</manifest>
```

4. Modifique el archivo **libs.versions.toml** para agregar las librerías necesarias para construir la app. Para ello, cambie la vista de su proyecto de **Android** a **Project**, posteriormente, localiza el archivo **libs.versions.toml** en la carpeta **gradle**.

- **Retrofit 2:** Librería encargada del consumo de las API.
- **Retrofit 2 Converter Gson:** complemento que simplificará el proceso de pasar un JSON a una clase Kotlin.

En la sección **[versions]**, defina las siguientes versiones de las herramientas.

- **retrofit = "2.9.0"**
- **gson = "2.9.0"**
- **okhttpLoggingInterceptor = "4.9.3"**

En la sección **[libraries]** defina las librerías con sus respectivos nombres y referencias a las versiones previamente escritas.

- **retrofit = { module = "com.squareup.retrofit2:retrofit", version.ref = "retrofit" }**
- **retrofit-converter-gson = { module = "com.squareup.retrofit2:converter-gson", version.ref = "gson" }**
- **okhttp-logging-interceptor = { module = "com.squareup.okhttp3:logging-interceptor", version.ref = "okhttpLoggingInterceptor" }**

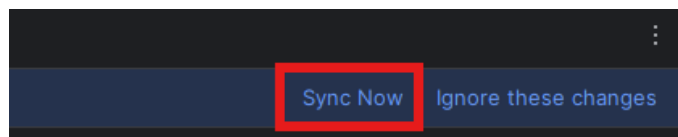
```
[versions]
agp = "8.11.2"
kotlin = "2.0.21"
coreKtx = "1.17.0"
junit = "4.13.2"
junitVersion = "1.3.0"
espressoCore = "3.7.0"
appcompat = "1.7.1"
material = "1.13.0"
activity = "1.11.0"
constraintlayout = "2.2.1"
retrofit = "2.9.0"
gson = "2.9.0"
okhttpLoggingInterceptor = "4.9.3"

[libraries]
androidx-core-ktx = { group = "androidx.core", name = "core-ktx", version.ref = "coreKtx" }
junit = { group = "junit", name = "junit", version.ref = "junit" }
androidx-junit = { group = "androidx.test.ext", name = "junit", version.ref = "junitVersion" }
androidx-espresso-core = { group = "androidx.test.espresso", name = "espresso-core", version.ref = "espressoCore" }
androidx-appcompat = { group = "androidx.appcompat", name = "appcompat", version.ref = "appcompat" }
material = { group = "com.google.android.material", name = "material", version.ref = "material" }
androidx-activity = { group = "androidx.activity", name = "activity", version.ref = "activity" }
androidx-constraintlayout = { group = "androidx.constraintlayout", name = "constraintlayout", version.ref = "constraintlayout" }
retrofit = { module = "com.squareup.retrofit2:retrofit", version.ref = "retrofit" }
retrofit-converter-gson = { module = "com.squareup.retrofit2:converter-gson", version.ref = "gson" }
okhttp-logging-interceptor = { module = "com.squareup.okhttp3:logging-interceptor", version.ref = "okhttpLoggingInterceptor" }
```

5. Modifique el archivo **build.gradle.kts (Module :app)** para agregar los nombres definidos previamente en el archivo **libs.versions.toml** para implementar las dependencias.

```
dependencies {  
  
    implementation(libs.androidx.core.ktx)  
    implementation(libs.androidx.appcompat)  
    implementation(libs.material)  
    implementation(libs.androidx.activity)  
    implementation(libs.androidx.constraintlayout)  
    testImplementation(libs.junit)  
    androidTestImplementation(libs.androidx.junit)  
    androidTestImplementation(libs.androidx.espresso.core)  
  
    // Retrofit, Gson, OkHttp  
    implementation(libs.retrofit)  
    implementation(libs.retrofit.converter.gson)  
    implementation(libs.okhttp.logging.interceptor)  
  
}
```

6. Sincronice el proyecto con Gradle dando clic en la opción **Sync Now** o en caso de que no aparezca, cierre y abra nuevamente el proyecto.



7. Agregue una nueva clase de Kotlin con el nombre **Alumno.kt** y modifique el código como se muestra a continuación.

```
data class Alumno(  
    val id: Int,  
    val nombre: String,  
    val apellido: String,  
    val edad: Int  
)
```

NOTA: Esta clase se utilizará para des serializar los objetos JSON que se reciben de la API.

8. Agregue una interfaz con el nombre **AlumnoApi** donde se definirán los métodos para llamar a la API.

```
import retrofit2.Call  
import retrofit2.http.*  
  
interface AlumnoApi {  
  
    @GET("escuela/alumno")  
    fun obtenerAlumnos(): Call<List<Alumno>>  
  
    @GET("escuela/alumno/{id}")  
    fun obtenerAlumnoPorId(@Path("id") id: Int): Call<Alumno>  
  
}
```

```

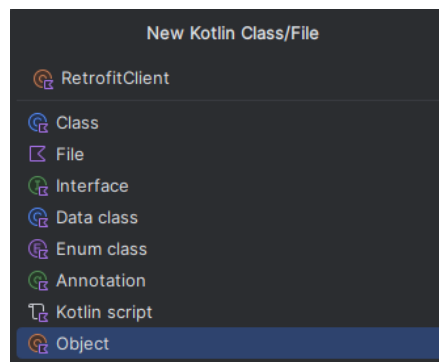
@POST("escuela/alumno")
fun crearAlumno(@Body alumno: Alumno): Call<Alumno>

@PUT("escuela/alumno/{id}")
fun actualizarAlumno(@Path("id") id: Int, @Body alumno: Alumno): Call<Alumno>

@PUT("escuela/alumno/{id}")
fun eliminarAlumno(@Path("id") id: Int): Call<Void>
}

```

9. Agregue un nuevo objeto con el nombre **RetrofitClient**, modifique el código según se muestra a continuación.



```

import retrofit2.Retrofit
import retrofit2.converter.gson.GsonConverterFactory

object RetrofitClient {
    private const val BASE_URL = "https://66240d4504457d4aaf9b8530.mockapi.io/api/"

    val instance: AlumnoApi by lazy {
        val retrofit = Retrofit.Builder()
            .baseUrl(BASE_URL)
            .addConverterFactory(GsonConverterFactory.create())
            .build()

        retrofit.create(AlumnoApi::class.java)
    }
}

```

10. Agregue una nueva clase con el nombre **AlumnoAdapter** que extienda de **RecyclerView.Adapter** para mostrar los datos en un **RecyclerView**.

```

import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.*
import androidx.recyclerview.widget.RecyclerView

class AlumnoAdapter(private val alumnos: List<Alumno>)

```

```

: RecyclerView.Adapter<AlumnoAdapter.ViewHolder>() {

    private var onItemClick: OnItemClickListener? = null

    class ViewHolder(view: View) : RecyclerView.ViewHolder(view) {
        val nombreTextView: TextView = view.findViewById(R.id.tvNombre)
        val apellidoTextView: TextView = view.findViewById(R.id.tvApellido)
        val edadTextView: TextView = view.findViewById(R.id.tvEdad)
    }

    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): ViewHolder {
        val view = LayoutInflater.from(parent.context)
            .inflate(R.layout.item_alumno, parent, false)
        return ViewHolder(view)
    }

    override fun onBindViewHolder(holder: ViewHolder, position: Int) {
        val alumno = alumnos[position]
        holder.nombreTextView.text = alumno.nombre
        holder.apellidoTextView.text = alumno.apellido
        holder.edadTextView.text = alumno.edad.toString()

        // Agrega el escuchador de clics a la vista del elemento de la lista
        holder.itemView.setOnClickListener {
            onItemClick?.onItemClick(alumno)
        }
    }

    override fun getItemCount(): Int {
        return alumnos.size
    }

    fun setOnItemClickListener(listener: OnItemClickListener) {
        onItemClick = listener
    }

    interface OnItemClickListener {
        fun onItemClick(alumno: Alumno)
    }
}

```

11. Agregue dos nuevos **Empty Views Activity** con los nombres **CrearAlumnoActivity** y **ActualizarAlumnoActivity**, modifique el código como se muestra a continuación.

CrearAlumnoActivity.kt

```

import android.content.Intent
import android.os.Bundle
import android.util.Log
import android.widget.Button
import android.widget.EditText

```

```

import android.widget.Toast
import androidx.activity.enableEdgeToEdge
import androidx.appcompat.app.AppCompatActivity
import androidx.core.view.ViewCompat
import androidx.core.view.WindowInsetsCompat
import retrofit2.Call
import retrofit2.Callback
import retrofit2.Response

class CrearAlumnoActivity : AppCompatActivity() {
    private lateinit var nombreEditText: EditText
    private lateinit var apellidoEditText: EditText
    private lateinit var edadEditText: EditText
    private lateinit var crearButton: Button

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContentView(R.layout.activity_crear_alumno)
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main)) { v, insets ->
            val systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars())
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom)
            insets
        }

        // Obtención de datos que envia actividad anterior
        val datos: Bundle? = intent.getExtras()

        nombreEditText = findViewById(R.id.editTextNombre)
        apellidoEditText = findViewById(R.id.editTextApellido)
        edadEditText = findViewById(R.id.editTextEdad)
        crearButton = findViewById(R.id.btnGuardar)

        crearButton.setOnClickListener {
            val nombre = nombreEditText.text.toString()
            val apellido = apellidoEditText.text.toString()
            val edad = edadEditText.text.toString().toInt()

            val alumno = Alumno(0,nombre, apellido, edad)

            // Obtiene la instancia de Retrofit con el cliente OkHttpClient
            val api = RetrofitClient.instance

            api.crearAlumno(alumno).enqueue(object : Callback<Alumno> {
                override fun onResponse(call: Call<Alumno>, response: Response<Alumno>) {
                    if (response.isSuccessful) {
                        Toast.makeText(this@CrearAlumnoActivity, "Alumno creado exitosamente", Toast.LENGTH_SHORT).show()
                        val i = Intent(getBaseContext(), MainActivity::class.java)

```

```

                startActivity(i)
            } else {
                val error = response.errorBody()?.string()
                Log.e("API", "Error crear alumno: $error")
                Toast.makeText(this@CrearAlumnoActivity, "Error al crear el
alumno", Toast.LENGTH_SHORT).show()
            }
        }

        override fun onFailure(call: Call<Alumno>, t: Throwable) {
            Toast.makeText(this@CrearAlumnoActivity, "Error al crear el
alumno", Toast.LENGTH_SHORT).show()
        }
    })
}
}
}

```

ActualizarAlumnoActivity.kt

```

package com.example.retrofitcrudapp

import android.content.Intent
import android.os.Bundle
import android.util.Log
import android.widget.Button
import android.widget.EditText
import android.widget.Toast
import androidx.activity.enableEdgeToEdge
import androidx.appcompat.app.AppCompatActivity
import androidx.core.view.ViewCompat
import androidx.core.view.WindowInsetsCompat
import com.google.gson.Gson
import com.google.gson.GsonBuilder
import com.google.gson.JsonSyntaxException
import org.json.JSONObject
import retrofit2.Call
import retrofit2.Callback
import retrofit2.Response
import retrofit2.Retrofit
import retrofit2.converter.gson.GsonConverterFactory

class ActualizarAlumnoActivity : AppCompatActivity() {

    private lateinit var api: AlumnoApi
    private var alumno: Alumno? = null

    private lateinit var nombreEditText: EditText
    private lateinit var apellidoEditText: EditText
    private lateinit var edadEditText: EditText

```

```

private lateinit var actualizarButton: Button

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    enableEdgeToEdge()
    setContentView(R.layout.activity_actualizar_alumno)
    ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main)) { v, insets ->
        val systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars())
        v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom)
        insets
    }

    nombreEditText = findViewById(R.id.nombreEditText)
    apellidoEditText = findViewById(R.id.apellidoEditText)
    edadEditText = findViewById(R.id.edadEditText)
    actualizarButton = findViewById(R.id.actualizarButton)

    // Crea una instancia de Retrofit con el cliente OkHttpClient
    val retrofit = Retrofit.Builder()
        .baseUrl("https://66240d4504457d4aaf9b8530.mockapi.io/api/")
        .addConverterFactory(GsonConverterFactory.create())
        .build()

    // Crea una instancia del servicio que utiliza la autenticación HTTP básica
    val api = retrofit.create(AlumnoApi::class.java)

    // Obtener el ID del alumno de la actividad anterior
    val alumnoId = intent.getIntExtra("alumno_id", -1)
    Log.e("API", "alumnoId : $alumnoId")

    val nombre = intent.getStringExtra("nombre").toString()
    val apellido = intent.getStringExtra("apellido").toString()
    val edad = intent.getIntExtra("edad", 1)

    nombreEditText.setText(nombre)
    apellidoEditText.setText(apellido)
    edadEditText.setText(edad.toString())

    val alumno = Alumno(0, nombre, apellido, edad)

    // Configurar el botón de actualización
    actualizarButton.setOnClickListener {
        if (alumno != null) {
            // Crear un nuevo objeto Alumno con los datos actualizados
            val alumnoActualizado = Alumno(
                alumnoId,
                nombreEditText.text.toString(),
                apellidoEditText.text.toString(),
                edadEditText.text.toString().toInt()
            )
        }
    }
}

```

```

    )
    //Log.e("API", "alumnoActualizado : $alumnoActualizado")

    val jsonAlumnoActualizado = Gson().toJson(alumnoActualizado)
    Log.d("API", "JSON enviado: $jsonAlumnoActualizado")

    val gson = GsonBuilder()
        .setLenient() // Agrega esta línea para permitir JSON malfor-
mado
        .create()

    // Realizar una solicitud PUT para actualizar el objeto Alumno
    api.actualizarAlumno(alumnoId, alumnoActualizado).enqueue(object :
Callback<Alumno> {
    override fun onResponse(call: Call<Alumno>, response: Res-
ponse<Alumno>) {
        if (response.isSuccessful && response.body() != null) {
            // Si la solicitud es exitosa, mostrar un mensaje de
éxito en un Toast
            Toast.makeText(this@ActualizarAlumnoActivity, "Alumno
actualizado correctamente", Toast.LENGTH_SHORT).show()
            val i = Intent(getBaseContext(), MainActi-
vity::class.java)
            startActivity(i)
        } else {
            // Si la respuesta del servidor no es exitosa, manejar
el error
            try {
                val errorJson = response.errorBody()?.string()
                val errorObj = JSONObject(errorJson)
                val errorMessage = errorObj.getString("message")
                Toast.makeText(this@ActualizarAlumnoActivity,
errorMessage, Toast.LENGTH_SHORT).show()
            } catch (e: Exception) {
                // Si no se puede parsear la respuesta del servi-
dor, mostrar un mensaje de error genérico
                Toast.makeText(this@ActualizarAlumnoActivity,
"Error al actualizar el alumno", Toast.LENGTH_SHORT).show()
                Log.e("API", "Error al parsear el JSON: ${e.mes-
sage}")
            }
        }
    }

    override fun onFailure(call: Call<Alumno>, t: Throwable) {
        // Si la solicitud falla, mostrar un mensaje de error en un
Toast
        Log.e("API", "onFailure : $t")
        Toast.makeText(this@ActualizarAlumnoActivity, "Error al ac-
tualizar el alumno", Toast.LENGTH_SHORT).show()

        // Si la respuesta JSON está malformada, manejar el error

```



```

        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main)) { v, insets ->
            val systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars())
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom)
            insets
        }

        val fab_agregar: FloatingActionButton = findViewById<FloatingActionButton>(R.id.fab_agregar)

        recyclerView = findViewById(R.id.recyclerView)
        recyclerView.layoutManager = LinearLayoutManager(this)

        // Obtiene la instancia de Retrofit con el cliente OkHttpClient
        api = RetrofitClient.instance

        cargarDatos(api)

        // Cuando el usuario quiere agregar un nuevo registro
        fab_agregar.setOnClickListener(View.OnClickListener {
            val i = Intent(getBaseContext(), CrearAlumnoActivity::class.java)
            startActivity(i)
        })
    }

    override fun onResume() {
        super.onResume()
        cargarDatos(api)
    }

    private fun cargarDatos(api: AlumnoApi) {
        val call = api.obtenerAlumnos()
        call.enqueue(object : Callback<List<Alumno>> {
            override fun onResponse(call: Call<List<Alumno>>, response: Response<List<Alumno>>) {
                if (response.isSuccessful) {
                    val alumnos = response.body()
                    if (alumnos != null) {
                        adapter = AlumnoAdapter(alumnos)
                        recyclerView.adapter = adapter

                        // Establecemos el escuchador de clics en el adaptador
                        adapter.setOnItemClickListener(object : AlumnoAdapter.OnItemClickListener {
                            override fun onItemClick(alumno: Alumno) {
                                val opciones = arrayOf("Modificar Alumno", "Eliminar Alumno")

                                AlertDialog.Builder(this@MainActivity)
                                    .setTitle(alumno.nombre)
                                    .setItems(opciones) { dialog, index ->

```

```

                when (index) {
                    0 -> Modificar(alumno)
                    1 -> eliminarAlumno(alumno, api)
                }
            }
            .setNegativeButton("Cancelar", null)
            .show()
        }
    })
}
} else {
    val error = response.errorBody()?.string()
    Log.e("API", "Error al obtener los alumnos: $error")
    Toast.makeText(
        this@MainActivity,
        "Error al obtener los alumnos 1",
        Toast.LENGTH_SHORT
    ).show()
}
}

override fun onFailure(call: Call<List<Alumno>>, t: Throwable) {
    Log.e("API", "Error al obtener los alumnos: ${t.message}")
    Toast.makeText(
        this@MainActivity,
        "Error al obtener los alumnos 2",
        Toast.LENGTH_SHORT
    ).show()
}

})
}

private fun Modificar(alumno: Alumno) {
    // Creamos un intent para ir a la actividad de actualización de alumnos
    val i = Intent(getBaseContext(), ActualizarAlumnoActivity::class.java)
    // Pasamos el ID del alumno seleccionado a la actividad de actualización
    i.putExtra("alumno_id", alumno.id)
    i.putExtra("nombre", alumno.nombre)
    i.putExtra("apellido", alumno.apellido)
    i.putExtra("edad", alumno.edad)
    // Iniciamos la actividad de actualización de alumnos
    startActivity(i)
}

private fun eliminarAlumno(alumno: Alumno, api: AlumnoApi) {
    Log.e("API", "id : $alumno")
    val llamada = api.eliminarAlumno(alumno.id)
    llamada.enqueue(object : Callback<Void> {
        override fun onResponse(call: Call<Void>, response: Response<Void>) {
            if (response.isSuccessful) {
                Toast.makeText(this@MainActivity, "Alumno eliminado",
                    Toast.LENGTH_SHORT).show()
            }
        }
    })
}

```

```

        cargarDatos(api)
    } else {
        val error = response.errorBody()?.string()
        Log.e("API", "Error al eliminar alumno : $error")
        Toast.makeText(this@MainActivity, "Error al eliminar alumno 1",
Toast.LENGTH_SHORT).show()
    }
}
override fun onFailure(call: Call<Void>, t: Throwable) {
    Log.e("API", "Error al eliminar alumno : $t")
    Toast.makeText(this@MainActivity, "Error al eliminar alumno 2",
Toast.LENGTH_SHORT).show()
}
})
}
}
}

```

13. Sobre el directorio **res/layout** agregue un nuevo archivo de recursos con el nombre **item_alumno.xml** y modifique el contenido según se muestra a continuación.

```

<?xml version="1.0" encoding="utf-8"?>
<androidx.cardview.widget.CardView
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_margin="8dp">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:padding="16dp">

        <ImageView
            android:id="@+id/ivAvatar"
            android:layout_width="64dp"
            android:layout_height="64dp"
            android:src="@drawable/avatar" />

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical"
            android:layout_marginStart="16dp">

            <TextView
                android:id="@+id/tvNombre"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:textSize="18sp"
                android:textStyle="bold" />

```

```

        <TextView
            android:id="@+id/tvApellido"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:textSize="16sp" />

        <TextView
            android:id="@+id/tvEdad"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:textSize="16sp" />

    </LinearLayout>

</LinearLayout>

</androidx.cardview.widget.CardView>

```

14. Modifique el layout **activity_main.xml** de la siguiente manera.

```

<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <com.google.android.material.floatingactionbutton.FloatingActionButton
        android:id="@+id/fab_agregar"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="bottom|end"
        android:clickable="true"
        android:contentDescription="TODO"
        app:fabSize="normal"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintHorizontal_bias="0.904"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintVertical_bias="0.902"
        app:srcCompat="@android:drawable/ic_input_add"
        tools:ignore="MissingConstraints" />

    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/recyclerView"
        android:layout_width="400dp"
        android:layout_height="match_parent"
        app:layoutManager="androidx.recyclerview.widget.LinearLayoutManager"

```

```

        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintHorizontal_bias="0.0"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintVertical_bias="0.0" />

</androidx.constraintlayout.widget.ConstraintLayout>

```

15. Modifique el código del layout **activity_crear_alumno.xml** y **activity_actualizar_alumno.xml**.

activity_crear_alumno.xml

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".CrearAlumnoActivity"
    android:orientation="vertical">

    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Agregar Nuevo Alumno"
        android:textSize="24sp"
        android:textStyle="bold"
        android:gravity="center" />

    <EditText
        android:id="@+id/editTextNombre"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Nombre del alumno"
        android:inputType="text" />

    <EditText
        android:id="@+id/editTextApellido"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Apellido del alumno"
        android:inputType="text" />

    <EditText
        android:id="@+id/editTextEdad"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Edad del alumno"
        android:inputType="number" />

```

```

    <Button
        android:id="@+id/btnGuardar"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Guardar" />

</LinearLayout>

```

activity_actualizar_alumno.xml

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".ActualizarAlumnoActivity"
    android:orientation="vertical">

    <EditText
        android:id="@+id/nombreEditText"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Nombre" />

    <EditText
        android:id="@+id/apellidoEditText"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="apellido" />

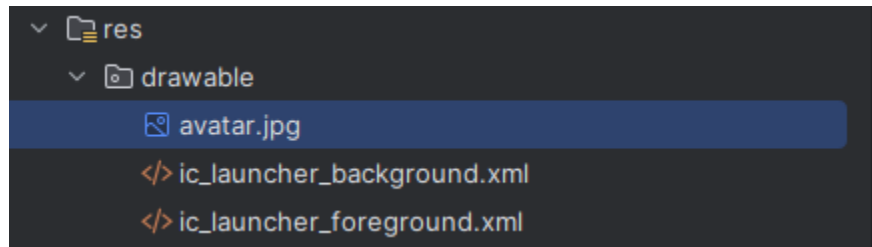
    <EditText
        android:id="@+id/edadEditText"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Edad" />

    <Button
        android:id="@+id/actualizarButton"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Actualizar" />

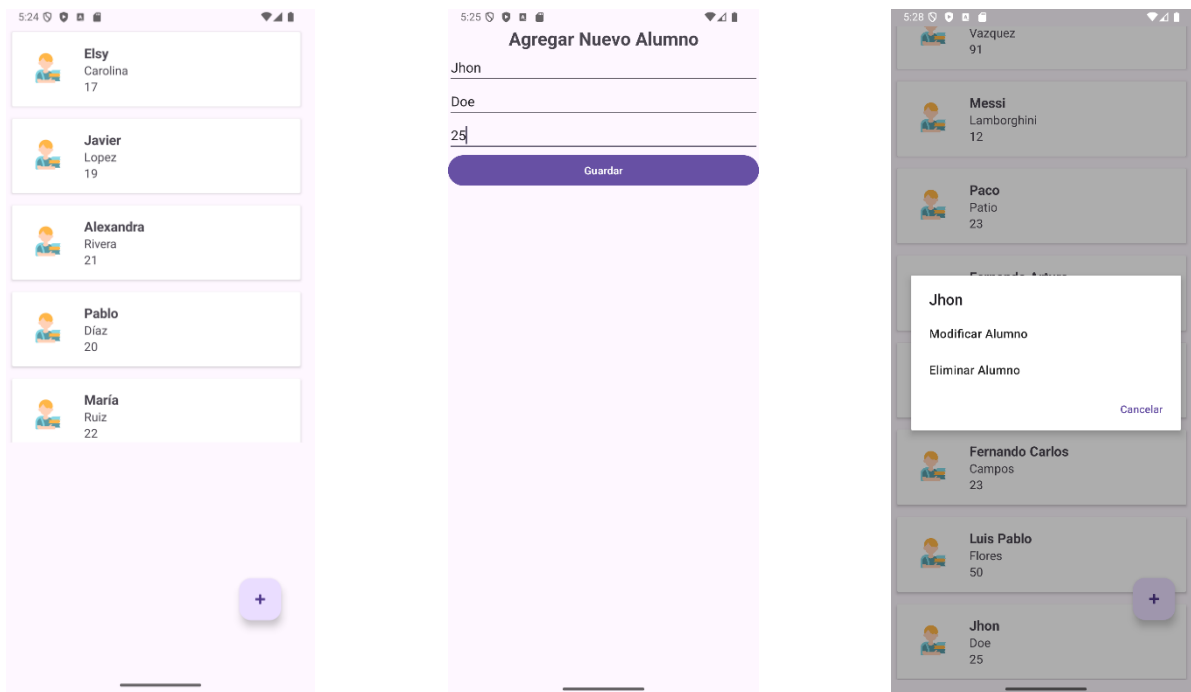
</LinearLayout>

```

16. Sobre el directorio **res/drawable** agregue un nuevo recurso con el nombre **avatar** (imagen disponible en el aula virtual).



17. Ejecute y compruebe el funcionamiento de la aplicación.



IV. DISCUSIÓN DE RESULTADOS

Dando continuidad al desarrollo del api REST y la aplicación móvil se le solicita:

- Agregar la tabla profesor (id, nombre, apellido, edad)
- Crear el api (/api/escuela/profesor)
- Dentro de la aplicación alumno agregar las pantallas para profesor.

V. BIBLOGRAFÍA

- Documentación Oficial Android
<https://developer.android.com/develop/ui/views/layout/recyclerview?hl=es-419#kotlin>
- Documentación Oficial Retrofit
<https://square.github.io/retrofit/>